UNICID
Universidade
Cidade de S. Paulo

Git e GitHub

Fundamentos de versionamento para desenvolvimento web

Parte 1 • Por que versionar?

Prof. Me. Gabriel Rosa Paz

Antes do Git: qual problema estamos tentando resolver?

Versionamento nasce da necessidade de controlar mudanças com segurança.

1 Sem histórico

Arquivos sobrescritos e dificuldade para recuperar uma versão que funcionava.

2 Sem autoria

Em equipes, fica difícil saber quem alterou cada parte e por quê.

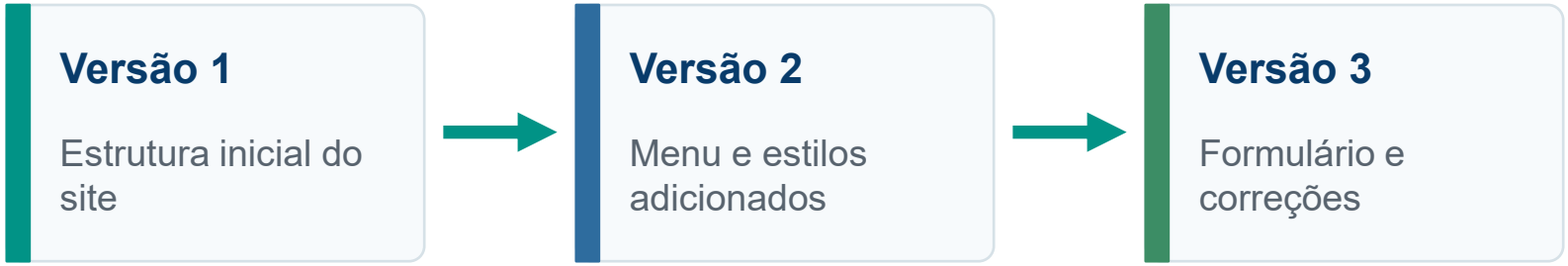
3 Sem colaboração

Duas pessoas editando o mesmo arquivo podem perder trabalho ou criar conflitos.

Controle de versão transforma mudanças em um histórico organizado, consultável e recuperável.

O que é controle de versão?

Um sistema que registra a evolução de arquivos ao longo do tempo.



Com um sistema de versionamento, cada ponto importante pode ser registrado. Depois, podemos comparar versões, identificar alterações e recuperar estados anteriores.

Por que o versionamento é importante?

Ele não serve apenas para “guardar cópias”: ele organiza o processo de desenvolvimento.

1. **Histórico**
   Registra o que mudou e quando.
2. **Recuperação**
   Permite voltar a um estado anterior.
3. **Comparação**
   Mostra diferenças entre versões.
4. **Colaboração**
   Organiza trabalho em equipe.
5. **Experimentação**
   Permite testar ideias em paralelo.
6. **Rastreabilidade**
   Relaciona mudança, autor e objetivo.

Uma analogia simples: “site_final_agora_vai.html”

Sem versionamento, cada pessoa inventa sua própria estratégia de cópias.

```
site.html
site_final.html
site_final2.html
site_final_corrigido.html
site_final_agora_vai.html
```



Com versionamento

Um único projeto
+
histórico de alterações
+
mensagens explicando cada mudança

Principais modelos de controle de versão

A evolução das ferramentas acompanha a necessidade crescente de colaboração.

1 Local

O histórico fica no próprio computador. Simples, mas limitado para equipes.

Local

2 Centralizado

Existe um servidor central. Os desenvolvedores dependem desse servidor para boa parte do fluxo.

CVCS

3 Distribuído

Cada desenvolvedor possui uma cópia completa do repositório e do histórico. Git é o exemplo mais conhecido.

DVCS

Exemplos de ferramentas de versionamento

Nem toda ferramenta funciona da mesma forma — e algumas continuam presentes em sistemas legados.

RCS / SCCS

Históricos locais e antigos.

Local

CVS

Modelo centralizado, muito usado historicamente.

Centralizado

Subversion (SVN)

Controle centralizado ainda encontrado em empresas.

Centralizado

Mercurial

Distribuído, semelhante em conceito ao Git.

Distribuído

Git

Distribuído, rápido e amplamente adotado.

Distribuído

Perforce Helix Core

Usado em cenários com grandes arquivos e equipes.

Centralizado/híbrido

Ferramenta local x serviço em nuvem

É importante separar o sistema de versionamento da plataforma que hospeda o projeto.

Ferramenta de versionamento

Executa no computador e controla o histórico do projeto.

Exemplos: Git, SVN, Mercurial.

Plataforma de hospedagem

Mantém repositórios remotos e adiciona recursos de colaboração, revisão e automação.

Exemplos: GitHub, GitLab, Bitbucket, Azure DevOps.

Git ≠ GitHub. Git é o sistema de versionamento; GitHub é uma plataforma que hospeda repositórios Git.

Onde o versionamento entra no fluxo de desenvolvimento?

Ele acompanha o projeto desde a primeira linha de código até a publicação.



Planejar	Editar	Testar	Versionar	Compartilhar	Revisar	Publicar
Definir tarefa	HTML, CSS e JS	Verificar no navegador	Commit da mudança	Push / Pull	Branches / PR	Deploy

Versionar é uma atividade contínua do desenvolvimento — não algo feito apenas no final do projeto.

Versionamento em projetos Front-End

O histórico pode registrar pequenas evoluções do site de forma clara.

```
index.html
style.css
script.js
img/
README.md
```

Commit 1
Cria estrutura HTML inicial

Commit 2
Adiciona identidade visual em CSS

Commit 3
Implementa menu responsivo

Commit 4
Corrige validação do formulário

“O objetivo não é decorar comandos. É compreender como registrar, compartilhar e recuperar a evolução do projeto.”

Conceito primeiro → prática depois

Fechamento da Parte 1

Antes de abrir o GitHub, precisamos levar quatro ideias para a prática.

1. Versão

Um estado identificável do projeto.

2. Histórico

Sequência organizada de mudanças.

3. Colaboração

Pessoas trabalham sem depender de cópias manuais.

4. Rastreabilidade

É possível entender quem mudou, o quê e por quê.

Na Parte 2: Git, GitHub, repositórios, commits, branches e colaboração.